

N-Body Simulation: Milestone 2

RISC-V Assembly Implementation

Hafsa Tanveer
IBA, SMCS
29041

Manal Fatima
IBA, SMCS
29018

Areeba Salman
IBA, SMCS
29075

I. INTRODUCTION

Milestone 2 translates the naive N-body simulation from Python into RISC-V assembly. The milestone involved three phases: refactoring the Python code into flat arrays compatible with assembly, implementing all leapfrog integration functions in scalar RISC-V assembly, and connecting the simulation to a Python visualizer through a memory-mapped file.

II. CODE REFACTORING

Assembly only supports flat arrays as its native data structure. There are no classes, structs, or dynamic lists. The Body class from Milestone 1 was therefore replaced with separate flat arrays for each physical attribute.

Listing 1. Refactored flat arrays

```
mass = [0.0] * N
x = [0.0] * N; y = [0.0] * N; z = [0.0] * N
vx = [0.0] * N; vy = [0.0] * N; vz = [0.0] * N
ax = [0.0] * N; ay = [0.0] * N; az = [0.0] * N
```

Array sizes are fixed at declaration and no `append()` calls are made, matching the static memory model of assembly. The refactored code was verified to produce identical output to the original class-based implementation.

In the assembly `.data` section, all arrays are declared with `.rept` directives and all float constants are stored as `.float` labels. The `.equ` directive cannot represent floating point values and cannot be used with `flw`, so constants such as `G` and softening require dedicated labels:

Listing 2. Float constant declarations

```
dt_float: .float 864000.0
half_dt_float: .float 432000.0
G_float: .float 6.67430e-11
softening_float: .float 1.0e9

px: .rept n
    .float 0
.endr # repeated for all arrays
```

III. ASSEMBLY IMPLEMENTATION

Since all physics values are real numbers, floating point instructions are used throughout. `flw/fsw` load and store floats, while `fmul.s`, `fadd.s`, `fsub.s`, and `fdiv.s` perform arithmetic. Array element addresses are computed as a byte offset of $i \times 4$ using `slli s2, s0, 2`, since each `.float` occupies 4 bytes. Because RISC-V does not permit two registers inside an address bracket, a separate add instruction precedes every load and store.

A. *calculate_drift* (Hafsa)

The drift step advances all positions by one full timestep using the current velocities: $x_i += v_i \cdot dt$

Listing 3. Drift function (abbreviated)

```
calculate_drift:
    la t0, px; la t3, vx
    la t6, dt_float
    flw ft0, 0(t6) # ft0 = dt
    li s0, 0; li s1, n
drift_loop:
    beq s0, s1, drift_end
    slli s2, s0, 2 # byte offset = i*4
    add s3, t0, s2
    flw ft1, 0(s3) # px[i]
    add s4, t3, s2
    flw ft2, 0(s4) # vx[i]
    fmul.s ft3, ft2, ft0
    fadd.s ft1, ft1, ft3
    fsw ft1, 0(s3) # px[i] += vx[i]*dt
    # repeated for py, pz
    addi s0, s0, 1
    j drift_loop
```

Testing was performed by linking with a C `printf` wrapper and comparing against Python reference values. Output matched within expected single-precision tolerance. A small precision difference of 25923.001953 vs 25923.0 was observed and confirmed as normal 32-bit floating point behaviour.

B. *calculate_kick* (Areeba)

The kick step applies a half-timestep velocity update using current accelerations: $v_i += 0.5 \cdot a_i \cdot dt$

Listing 4. Kick function (abbreviated)

```
calculate_kick:
    la t0, vx; la t3, ax
    la t6, half_dt_float
    flw ft0, 0(t6) # ft0 = 0.5*dt
    li s0, 0; li s1, n
kick_loop:
    beq s0, s1, kick_end
    slli s2, s0, 2
    add a0, t0, s2
    flw ft1, 0(a0) # vx[i]
    add a1, t3, s2
    flw ft2, 0(a1) # ax[i]
    fmul.s ft3, ft2, ft0
    fadd.s ft1, ft1, ft3
    fsw ft1, 0(a0) # vx[i] += ax[i]*half_dt
    # repeated for vy, vz
    addi s0, s0, 1
    j kick_loop
```

C. calculate_acc (Manal)

The force calculation uses a nested loop over all N^2 body pairs. Accelerations are first zeroed using `fmv.w.x` to produce a correct float zero. For each pair (i, j) where $i \neq j$, the displacement, squared distance, and softened distance cubed are computed. The softening term prevents division by zero for close encounters:

$$r_{\text{soft}}^3 = \sqrt{r^2 + \epsilon^2} \cdot (r^2 + \epsilon^2) \quad (1)$$

Listing 5. Force calculation (abbreviated)

```
calculate_acc:
# G and softening^2 loaded into fs0, fs1
# (saved registers survive the inner loop)
la a0, G_float
flw fs0, 0(a0)
la a0, softening_float
flw fs1, 0(a0)
fmul.s fs1, fs1, fs1 # precompute softening^2
outer_loop:          # i = 0 to n
fmv.w.x ft0, zero    # reset ax[i]=ay[i]=az[i]=0
...
inner_loop:          # j = 0 to n
beq s0, s3, skip_self # skip i==j
# compute dx,dy,dz with fsub.s
# dist_sq = dx*dx + dy*dy + dz*dz
fadd.s ft6, ft6, fs1 # add softening^2
fsqrt.s ft7, ft6     # sqrt
fmul.s ft7, ft7, ft6 # dist_soft^3
fdiv.s ft9, ft9, ft7 # acc_factor=G*mj/dist^3
# ax[i] += acc_factor * dx (and ay, az)
```

G and softening must be loaded via their `.float` labels into saved float registers `fs0` and `fs1` so they survive across all inner loop iterations without being reloaded. Registers `t7–t9` do not exist in RISC-V; `a0` and `a1` are used instead as temporary address registers.

IV. MEMORY MAPPING INTERFACE

The simulation communicates position data to the Python visualizer through a memory-mapped file named `shared.mem`. The file layout is:

Offset	Contents
0	px array ($N \times 4$ bytes)
$N \times 4$	py array ($N \times 4$ bytes)
$2 \times N \times 4$	pz array ($N \times 4$ bytes)
$3 \times N \times 4$	sync flag (4-byte int)

After each leapfrog step, the assembly writes `px`, `py`, and `pz` sequentially to the mapped region using `memcpy` via a C helper function. A synchronization flag at the end of the region coordinates access between the two processes:

Listing 6. Synchronization protocol

```
// Assembly side (csv_loader.c helper)
while (*flag == ASSEMBLY_WROTE) { } // wait for Python
memcpy(base, px, N*sizeof(float));
memcpy(base+N, py, N*sizeof(float));
memcpy(base+2*N, pz, N*sizeof(float));
*flag = ASSEMBLY_WROTE; // signal Python

// Python side (visualizer.py)
if flag == ASSEMBLY_WROTE:
    px = read N floats from offset 0
    py = read N floats from offset N*4
    pz = read N floats from offset 2*N*4
    flag = PYTHON_READING # signal assembly
```

This ensures the assembly never overwrites data while Python is still reading, and Python never reads a partially written frame. The visualizer renders positions using a perspective projection with camera rotation controlled by the arrow keys.

V. QEMU PLUGIN ANALYSIS

QEMU user-mode emulation supports instrumentation plugins for performance profiling. The following plugins are available as documented in the RISC-V development guide:

Plugin	Purpose
libinsn.so	Count total instructions
libmem.so	Track memory reads/writes
libexeclog.so	Log every instruction
libhotpages.so	Identify hot memory pages

Listing 7. Running with a plugin

```
qemu-riscv64 -plugin ./myplugin.so ./nbody
```

Since the simulation loops indefinitely, a `j end` branch is inserted to terminate after one iteration before profiling runs. The instruction count plugin is particularly relevant for comparing the $O(N^2)$ naive implementation against the $O(N \log N)$ Barnes-Hut algorithm in a future milestone.

VI. CHALLENGES FACED

Hafsa (Drift): Floating point in assembly required learning separate `flw/fsw` and arithmetic instructions. Linking with a C `printf` wrapper for debugging and resolving the missing shared library path via `-L /usr/riscv64-linux-gnu` were non-obvious hurdles. The invalid `flw ft1, 0(t0+s2)` syntax was a key learning point since RISC-V requires a separate `add` before any two-register memory access.

Areeba (Kick): Correctly initializing loop counters required `li` rather than the two-operand `addi` that was initially used. Tracking which of the six array base addresses was in which register across the three velocity components required careful register documentation.

Manal (Force Calculation): Managing two simultaneous loop counters in the nested loop, resetting accelerations with `fmv.w.x` rather than copying an existing register, loading G and softening from `.float` labels rather than `.equ` constants, and working around the non-existence of `t7–t9` were the primary challenges in the most complex function of the simulation.

VII. CONCLUSION

This milestone successfully translated all three leapfrog integration functions of the naive N-body simulation into scalar RISC-V assembly. The drift and kick functions were verified against Python reference outputs. The force calculation handles all N^2 gravitational interactions with correct softening. Position data is shared with a Python visualizer through a synchronized memory-mapped file, completing the full simulation pipeline from assembly physics to real-time visual output.